AgentWorkShop: A Task-to-Device Integration Architecture for Industrial Agents

***Abstract*—Industrial-agent tasks need an execution path that carries production context from collaboration services to device-facing operations and keeps command results distinct from later process observations. AgentWorkShop provides a modular task-to-device integration architecture that connects workspace and task services, registered agent-harness adapters, line–product–recipe–run context, per-call dispatch checks, semantic acquisition and control nodes, and configured protocol drivers. Prompt context is assembled for each agent turn; the scoped tool catalog is provisioned with the registered harness, and operation authority is checked again for each call. Route handlers retain available command, approval, observation, and recipe-version records in their owning services. We evaluate two simulator paths in cast-film and annealing scenes. In R20, six sequential tasks through one registered harness complete with synthetic approval and verified readback, although all six post-task samples remain outside target bands. The exploratory R13 application run records protocol access, readback, and data lineage, while both model candidates fail the configured gates; its exact runtime source was not pinned. These results establish the tested integration paths, but not process attainment or configuration-reuse benefit.**

***Index Terms*—Industrial agents, industrial communication, industrial informatics, production systems, semantic data integration.**

I. Introduction

INDUSTRIAL-AGENT deployments must connect each task to its production context, process signals, and equipment interfaces. Existing work addresses end-to-end automation control, operator-reviewed control-sequence generation, modular factory skills, and production scheduling [1]–[4]. Other studies target PLC code generation, context-aware control, resource capability discovery, shop-floor coordination, digital-twin fault handling, natural-language machine access, and robotic-line control [5]–[10]. Across production lines, agent roles, signal maps, protocols, objectives, and product/run context vary. An integration architecture must accommodate these differences around shared task and operation services while keeping each action tied to its active production context.

Consider two production scenes with different process signals and interfaces: cast-film thickness control and annealing-zone temperature control. An agent task in either scene needs production context, typed signal access, and a route from a named operation to the configured device protocol. The framework must also distinguish a command receipt from a subsequent process observation. The shared task and operation services can therefore be configured with scene-specific signal maps, protocols, and limits while retaining a common execution path. The evaluation instantiates this path in two simulator scenes.

This paper presents AgentWorkShop as a task-to-device integration architecture and implementation for industrial agents. It connects collaborative tasks and registered harnesses to production context, manufacturing execution system (MES) data, analysis and modeling services, data-acquisition (DAQ) and write-control (DCW) nodes, and protocol-specific drivers. Its access surfaces include command-line, web, SDK, REST, MCP, and A2A interfaces. Task and chat routes enter the collaboration runtime, whereas a direct REST tool-invocation route can call the shared operation dispatcher without creating a task. Scene blueprints define signal maps, objectives, and guards; channel profiles, tool policies, and agent bindings scope operations. Agents and callers invoke named operations through shared runtime checks; the framework does not choose routes autonomously and retains proposals, command confirmation, and process observations as distinct records. Fig. 1 presents the system architecture, and Fig. 2 shows the cross-module task flow. Section IV details the node-exploration and recipe lifecycles.

The paper makes three contributions:

1) It specifies a cross-layer task-to-operation contract that assembles prompt-time production context, supplies a scoped catalog to a registered harness, rechecks each returned call, routes semantic process nodes to configured device drivers, and preserves available command and process records by route.
2) It documents an implementation architecture that composes collaboration tasks, production-context binding, operation checks, semantic I/O, and service-owned records. The contribution is this implemented integration path; it does not introduce a new agent-reasoning or process-control algorithm.
3) It evaluates two simulator paths across cast-film and annealing scenes. R20 records six completed, synthetically approved, readback-verified task executions with one run-tagged sample per task, all outside target bands. Exploratory R13 records protocol and data-path checks while both model candidates fail their configured gates. These results characterize the tested integration paths rather than process-control efficacy.

Section II reviews related industrial-agent and integration work. Section III presents the design rationale, architecture, and cross-module execution flow; Section IV explains module responsibilities and their integrated operation. Section V describes the experiments and results, and Sections VI and VII discuss the evidence limits and conclude the paper.

II. Related Work

This section relates AgentWorkShop to plant-facing industrial agents and to production and automation abstractions for context, tool access, and operational constraints. Table I compares selected systems that address control generation,

TABLE I
SELECTED PLANT-FACING AGENT SYSTEMS AND THEIR INTEGRATION SETTINGS. THE TABLE COMPARES THE STATED EXECUTION SCOPE AND EVALUATION FOCUS; IT IS NOT A PERFORMANCE RANKING AND DOES NOT INFER UNREPORTED FEATURES.

System	Execution unit	Stated integration boundary	Evaluation focus
Huang et al. [2]	Automated industrial control generation	Multimodal industrial CPS architecture; the paper's stated focus is generating control actions	Control generation is its stated scope; no head-to-head comparison is made here
Köhle et al. [3]	Factory modules; orchestrator, peer-to-peer, and monolithic designs	Per-module MCP servers expose OPC UA methods, with MQTT coordination, injected factory state, and state-based tool filtering	Nine simulated production challenges, including silent-fault detection
Vieira da Silva et al. [12]	LLM planning and execution of manufacturing steps	Manufacturing resource functions exposed through MCP	Laboratory-scale multi-step process with constraint handling
Köhle et al. [13]	AI-assisted production-recipe composition	Reusable semantic recipes composed from OPC UA skills and validated by virtual commissioning	Proof-of-concept assembly workstation represented in Unity
Biskupovic et al. [11]	Service selection and reconfiguration in service-oriented ICPSs	Predefined component library; performance and computational-load criteria; online monitoring	Reselection after poor performance; reported adaptability and performance results
AgentWorkShop (this work)	Collaboration task and registered harness call	Line–product–recipe–run context; per-call policy checks; semantic process nodes resolve to configured protocol drivers	Two configured simulator scenes exercise the task-to-device path; command receipts and process observations remain distinct, context-linked records

factory-module coordination, MCP-based resource access, semantic recipes, and supervisory service reconfiguration. It uses execution scope, stated integration boundary, and evaluation focus as common dimensions. This is a focused comparison rather than an exhaustive survey, and it does not treat an unreported feature as absent.

A. Industrial Agents and Plant Integration

Industrial-agent systems span end-to-end automation control, supervisory services, programmable logic controller (PLC) interfaces, manufacturing coordination, and operator-facing machine access. Xia et al. present an LLM-agent framework for end-to-end industrial automation control, using structured prompts and event-driven context for production planning and operation [1]. Huang et al. present a multimodal architecture whose stated focus is automated control generation for industrial cyber-physical systems [2]. Biskupovic et al. study agent-based supervision for service-oriented industrial systems [11]; Agents4PLC and InstructMPC address PLC code generation and human–LLM context-aware control, respectively [5], [6]. Manufacturing studies consider dynamic resource-capability exploration [7], multi-agent shop-floor coordination [8], and digital-twin-supported fault handling [9]. Natural-language machine access [10] and real-time LLM control of a robot-operated serial line [4] extend this work to operator interfaces and robotic-line control. Together, these studies address different operational contexts, interfaces, and allocations of responsibility between agents, operators, and controllers.

These works demonstrate interfaces and workflows at adjacent layers. Vieira da Silva et al. expose manufacturing resource functions through MCP and evaluate LLM-driven multi-step planning with constraints; Köhle et al. compose reusable OPC UA skills into semantic recipes and validate them through virtual commissioning [12], [13]. Köhle et al. also examine MCP-based coordination across factory modules, while Huang et al. focus on control generation and Biskupovic et al. on service reconfiguration [2], [3], [11]. AgentWorkShop's contribution is an implemented integration architecture that composes collaboration tasks and registered harnesses with bound production context, shared per-call checks, semantic I/O, and route-specific records. On the task path, available line/run context grounds an agent turn; a returned operation passes current dispatch checks before the configured node driver is selected, while its tool result, command receipt, and subsequent process observation remain distinct. The paper claims this cross-layer composition and its tested software paths; it does not claim a new protocol, recipe formalism, agent-reasoning method, or process-control algorithm. No head-to-head comparison with the cited systems is made. Authority and command confirmation are operational services consistent with safety-aware decision support [14]; they do not replace process protection or establish operator effectiveness.

B. Production Context, Agent Interfaces, and Automation

Industrial integration also depends on abstractions that are not specific to agent architectures. MCP standardizes tool discovery and calls with request-scoped authorization [15], whereas ISA-88 and ISA-95 describe recipes and production operations [16], [17]. Run-to-run methods connect process observations to successive production runs [18]. At the agent level, AgentSpec and CaMeL address tool behavior and capability-based information flow [19], [20]. Together, these works provide complementary abstractions for production context, tool interaction, and information flow. AgentWorkShop composes such abstractions with its task runtime, operation checks, semantic I/O, and route-specific records; this paper

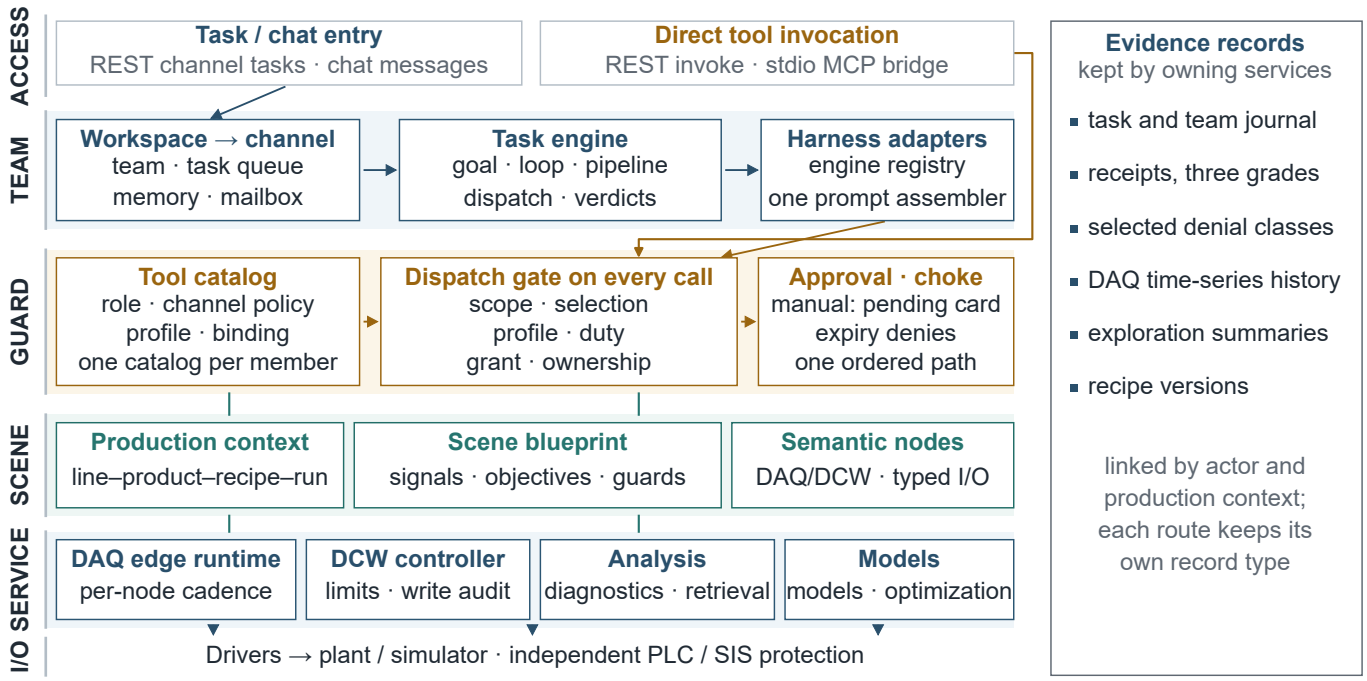


Fig. 1. AgentWorkShop software architecture. Access, collaboration, governance, production context, services, and field I/O form six connected bands. Registered harness and driver interfaces expose extension points; route-specific records remain in their owning services and are linked by available actor and production-context identifiers. The field interface ends at the registered drivers; PLC and safety-instrumented protection remain external.

evaluates selected paths through that composition rather than proposing a replacement for the underlying standards.

Industrial security and record requirements further constrain that architecture: FDA guidance discusses trustworthy records and audit trails [21], IEC 62443-3-3 specifies system-security requirements [22], and DNP3 includes select-before-operate [23]. Control-assurance methods address state evolution through shielding, control-barrier functions, or Simplex architectures [24]–[26]. These standards and methods inform the boundaries within which an industrial-agent framework must operate; they are not contributions claimed by AgentWorkShop.

The evaluation tests selected software paths and simulator scenes, not physical-plant performance.

III. System Design

AgentWorkShop is designed for industrial tasks whose agents, production contexts, interfaces, and process signals vary between deployments. Its architecture organizes these variations around shared task and operation services, checks each proposed operation against the active production context before device routing, and retains route-specific evidence that distinguishes a command receipt from a measured process response.

A. Design Rationale and Requirements

Five requirements—scene variability, harness variability, operation authority, outcome observability, and extensibility—organize six design rules (D1–D6). These are deployment constraints drawn from the target workflows. D1 stores scene settings as versioned data; D2 provides registered harness, tool, and driver interfaces; D3 separates proposal from execution; D4 treats the tool catalog as discovery rather than permission; D5 routes actuator writes through the common DCW admission point; and D6 retains evidence by operation.

The requirements trace to implementation as follows. Scene variability maps to D1, scene blueprints, and context resolution; harness variability and extensibility map to D2, the registered harness and driver interfaces; operation authority maps to D3–D5, the shared dispatcher, grants, bindings, and write admission; outcome observability maps to D6, route-specific receipts, and observations. R13 exercises two scene-specific protocol and data paths; R20 exercises the configured task-to-device path through one registered harness, synthetic approvals, and route-specific records. Section V reports the experimental scope and evidence limits. AgentWorkShop supplies task and operation services; agent strategy and independent process protection remain outside this design.

B. Layered Architecture

Fig. 1 maps the reference implementation onto six bands and their route-specific evidence records. The access band exposes command-line, web, SDK, REST, MCP, and A2A entry surfaces. Task and chat routes enter the collaboration band, which contains workspaces, channels, task queues, shared memory, mailboxes, task templates, and harness adapters. A direct REST tool-invocation route can call the shared operation dispatcher without creating a task or invoking a harness; MCP also includes separate management and task operations. The governance band projects an operation catalog and checks each invocation. The context band assembles bound-line and active-run information for prompt grounding, while operation handlers resolve the live state relevant to their route. The service band provides acquisition, write control, analysis,

retrieval, and optional model services. The field-I/O band resolves semantic nodes to built-in or plugin drivers for a plant or simulator.

Each band has a distinct responsibility. Task/chat routes enter the task runtime; direct operation routes enter the dispatcher; and both use shared operation checks where applicable. A harness adapter supplies agent-specific interaction on task paths, semantic nodes hide protocol details from operations, and the driver registry provides the device-facing extension point. Owning services retain task, command, observation, and version records separately and link them to available actor and production context. No single status conflates these outcomes.

C. Cross-Module Execution Flow

Fig. 2 summarizes a task turn from entry through operation execution, with route-specific evidence retained by owning services. Registered harness adapters receive a scoped tool catalog during setup and supported registry refreshes; the runtime assembles prompt-time production context for each agent turn. This context is grounding information, not a task-pinned transaction. For each returned tool call, the shared dispatcher rechecks invocation scope and channel policy and, for industrial operations, current grant and binding conditions before handler selection. The selected handler then applies route-specific guards, active-run checks, approval revalidation, and device-I/O steps as applicable. Command receipts and later process observations remain distinct records.

For an industrial operation u by agent a in channel c, define the shared dispatch condition as

$$\mathcal{A}(a, c, u) = S(a, c, u) \wedge P(c, u) \wedge G(a, u) \wedge B(a, u), \quad (1)$$

where S, P, G, and B denote invocation scope, current channel policy, runtime grant, and agent binding/resource ownership, as applicable to the operation. The policy term includes the applicable channel tool selection, profile, and duty-separation checks. This condition is necessary for handler selection, not sufficient for actuation: the selected route applies its own guards and approval, then rechecks authority and applicable limits before dispatch.

Fig. 3 gives pseudocode for these cross-module contracts; it is a reference flow rather than one literal orchestrator function. Route-specific timing, approval, and evidence rules are detailed in Section IV.

IV. Module Functions and Integration

The architecture in Fig. 1 is implemented as cooperating modules rather than a single monolithic agent. Shared interfaces connect task execution to configuration, industrial services, protocol drivers, and route-specific evidence. This section describes those responsibilities and the checks that preserve their boundaries.

The extension contracts expose the data passed at each boundary. A harness call is normalized as `HostToolCall(toolName, arguments)` and returns `HostToolResult(text, isError)`. A DCW driver accepts an engineering value, tolerance, range, and driver configuration; its result separates write status, raw value, readback, and acknowledgment grade (readback-verified, transport-ack, or unverified). Each DAQ sample stores its node, timestamp, value, and state, with optional line, product, recipe, and run IDs. A write anchor records line/node, prior and requested values, source, actor, time, and optional run/task/approval IDs; recipe definitions retain separate versions. These fields keep tool errors, device receipts, observations, and production lineage inspectable as separate records.

A. Task Runtime and Harness Adapters

Task and chat access surfaces create or join a workspace channel, where the task engine manages members, task queues, shared memory, and execution templates. A direct REST tool-invocation endpoint can enter the shared dispatcher without creating a task or invoking a harness; the stdio MCP operation bridge uses this route, while other MCP operations expose management and task services. The harness registry instantiates an adapter behind a common agent interface; the prompt builder combines the task with the active scene brief, industrial context, channel profile, agent binding, and retrieved memory. Tool calls return to the shared dispatcher regardless of which registered harness produced them. The registry is an extension point; the experiments do not establish portability across harnesses. Industrial context is assembled for each agent turn rather than pinned for the task lifetime. Operation handlers check current route-specific state at invocation; recipe writes recheck the binding, authorization, and active run after any human-approval wait. The implementation does not compare the complete prompt-time line–product–recipe–run snapshot with a task-pinned context version, so the prompt snapshot is grounding information, not a transactional authorization token.

B. Production Context and Configuration

Scene blueprints describe devices, signal maps, objectives, and process guards. A task resolves these definitions against its active line–product–recipe–run context. The channel profile selects the available capability family, the tool policy narrows the operation catalog, and the agent binding scopes the nodes, controls, and recipe operations available to that member. These filters can restrict one another but cannot broaden the underlying device range. The dispatcher re-reads policy at execution, so a cached catalog does not retain authority after a grant or policy change. The prompt builder assembles the agent's available bound-line and active-run context on each turn; it does not pin the complete context tuple to the task. Specific production snapshot, trial, and MPC routes, together with Hybrid Twin training submissions, resolve and validate a versioned `SceneContract` at operation time. The Twin physics-specification draft route can instead accept a caller-supplied scene description; SceneContract enforcement is therefore route-specific and does not validate or pin all prompt context.

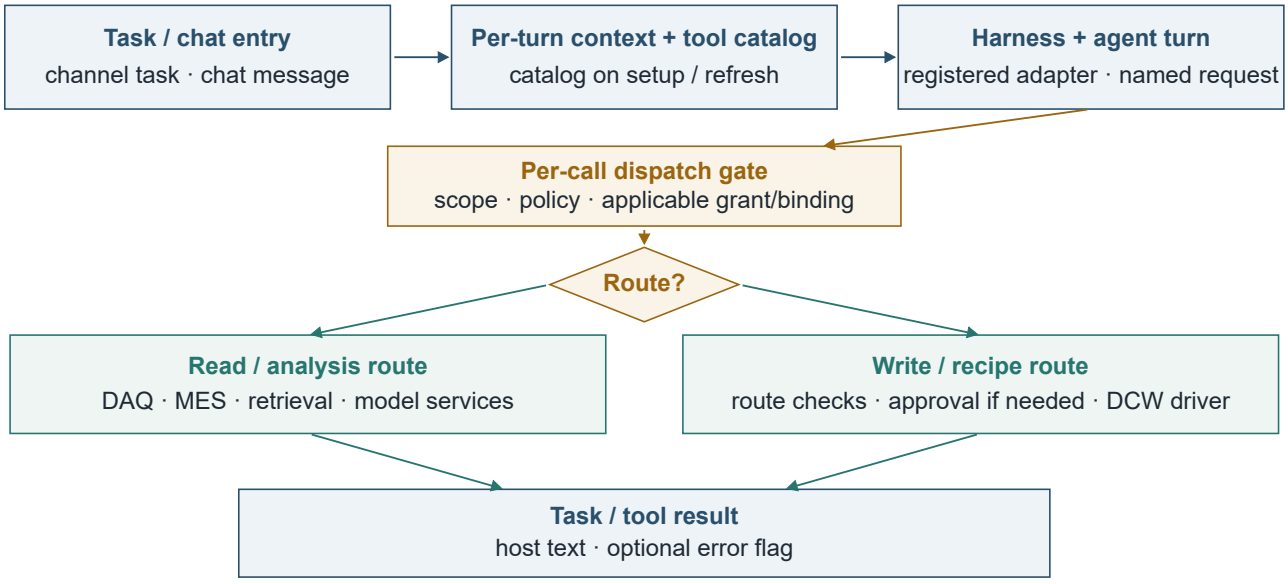


Fig. 2. Cross-module task execution in AgentWorkShop. The registered harness uses a scoped catalog provisioned at setup and supported registry refreshes; the runtime assembles prompt context per agent turn. The dispatcher checks each returned call for scope, channel policy, and applicable grant and binding conditions before handler selection. Route handlers resolve active-run and approval-dependent state as applicable. The shared tool result contains host text and an optional error flag; route records remain with owning services. Prompt context is not a transactional authorization token. Dispatch-gate errors return before handler execution, whereas route handlers may read state before refusing a write. Durable denial auditing is selective. Receipts and later observations remain distinct.

Require: task q, channel c, bound agent a, registered scoped catalog T
Ensure: task result; any retained evidence remains with its owning service
$x_t \leftarrow \textsc{ResolvePromptContext}(c, q)$
$m \leftarrow \textsc{RetrieveMemory}(q)$
$p \leftarrow \textsc{AssemblePrompt}(q, x_t, m)$
$h \leftarrow \textsc{InvokeRegisteredHarness}(a, p, T)$
for all tool request u returned by h **do**
  $g \leftarrow \textsc{CheckScopeAndChannelPolicy}(a, c, u)$
  if not g **then**
    $\textsc{ReturnToolError}$(h,u); **continue**
  end if
  if u is an industrial operation **then**
    $v \leftarrow \textsc{CheckGrantBindingOwnership}(a, u)$
    if not v **then**
      $\textsc{ReturnToolError}$(h,u); **continue**
    end if
  end if
  $k \leftarrow \textsc{SelectHandler}(u)$
  $r \leftarrow \textsc{ExecuteRouteHandler}(k, a, c, u)$ ▷ handler resolves live state and may persist route-specific records
  $\textsc{ReturnDeclaredToolResult}$(h,r)
end for
return task state and tool results

Fig. 3. Reference pseudocode for cross-module execution in AgentWorkShop. Evidence persistence is route-specific and remains with the owning service; the shared tool result does not carry a generic evidence envelope.

C. Acquisition, Analysis, and Modeling Services

Semantic DAQ nodes expose typed values, engineering units, ranges, and sampling settings while hiding protocol-specific reads. A per-node DAQ runtime schedules acquisition and stores timestamped observations; the gateway aggregates node pipelines. MES-facing operations provide the configured production-data path. Analysis and retrieval services consume these records, while optional model, training, optimization, and digital-twin services are invoked through their registered operation handlers. These services complement the agent workflow; AgentWorkShop does not prescribe a single optimization strategy.

D. Semantic I/O and Actuation

The driver registry resolves a semantic node to a built-in or plugin driver. DAQ handlers use the node's acquisition interface; DCW handlers route a write or readback through the controller and the configured protocol driver. This separation lets recipes and agents refer to semantic controls without embedding Modbus, OPC UA, MQTT, HTTP, or MES transport details in their operation arguments. The supported driver mappings are implementation interfaces; only the mappings and workflows enumerated in Section V are experimentally exercised.

E. Shared Dispatch and Write Admission

Every named operation passes the dispatcher. Its ordered checks validate invocation scope and current channel policy (tool selection, profile, and applicable duty separation), then

the agent's binding and runtime grant and ownership of referenced resources. A refusal returns the excluding condition. Denial persistence is selective: grant denials and selected resource-reference denials have explicit audit paths, whereas scope and channel-policy denials are not uniformly retained as durable audit events. A catalog entry therefore describes availability but does not authorize execution. Handlers for operations that require manual approval suspend for a decision, then revalidate authority and active limits before dispatch.

All actuator writes converge on the DCW admission point. For an online node write, the admissible interval is

$$\mathcal{I}_n^{\text{online}}(t) = \mathcal{H}_n \cap \mathcal{B}_n \cap \mathcal{P}_n(t) \cap \mathcal{R}_n(t), \tag{2}$$

where $\mathcal{H}_n$ is the hard engineering range and $\mathcal{B}_n$, $\mathcal{P}_n$, and $\mathcal{R}_n$ are the baseline, active-product, and active-recipe limits. Missing optional limits impose no restriction, whereas an empty intersection rejects the request and identifies the excluding layer. Recipe dispatch applies the hard, baseline, and product limits to the replacement recipe rather than intersecting with the previous recipe window. The actuator path also enforces step, interval, and hold-window checks. These admission rules constrain commands; process objectives and coupled variable guards are assessed from later observations. The framework does not replace independent PLC or safety-instrumented protection.

F. Integrated Operation and Route-Specific Evidence

A recipe trial illustrates how the modules cooperate. The task runtime delivers a named `recipe_trial` call from the agent harness; the dispatcher checks the agent's recipe binding and execution scope; the recipe handler checks the active run, candidate parameters, and route timing; and a manual binding requests approval before the DCW controller sends the batch through its driver. The handler returns a per-parameter receipt. The caller then obtains DAQ observations and can request a separate update, apply, or rollback operation. A trial receipt confirms command handling; it does not by itself establish process attainment or persist a new recipe version.

The routes retain different records because they answer different questions. A driver acknowledgment reports command evidence; DAQ reports a process observation; an exploration summary records a bounded probe; and a recipe version records a saved definition. Their relation through actor and production context allows an execution to be inspected without treating these outcomes as interchangeable.

G. Route-Specific Lifecycles and Evidence

a) Timing and approval: Timing guards are route-specific. Online node writes enforce an effective minimum spacing of 60 s. Recipe dispatch uses a per-recipe interval anchored to approval, while trials also use a configurable per-line cadence (default 300 s). The recipe interval can refuse a proposal before an approval request is created. Checks occur at proposal or execution according to the operation, and experiment-specific pacing overrides are reported with the corresponding results.

Manual bindings create an approval request containing the actor, node or recipe, candidate values, applicable limits, expiry, and parameter bases. The default wait expires after 180 s and refuses execution. After approval, the executor rechecks authorization and active limits to account for context changes during the wait. For `recipe_apply`, a changed definition requires a new proposal; an unchanged approval dispatches a copy of the approved parameter snapshot. A line-level recipe mode defaults to manual; changing it to automatic requires administrator confirmation. The approval record authorizes a workflow action and does not replace process protection.

b) Node probe: The bounded node probe is exposed as `optimization_explore` to the Auto Modeling Lab (AML) optimization profile (`aml_optimization`). The profile validates a bound control and target DAQ scope, captures a starting control and target baseline, invokes the internal DCW write path, then waits 10–300 s (default 45 s) and queries a trailing 45-s target window before saving an exploration summary. Since each window is relative to its query time, approval can separate baseline sampling from actuation; a 10-s wait can leave about 35 s of pre-write samples in the later window. The command receipt and target summary are therefore returned separately. A successful changed-value write opens a node-optimization record; refusal or expired approval creates a rejected summary without a post-write sample.

c) Recipe operations: `recipe_trial` checks the bound recipe and active run, then dispatches candidate overrides as a batch with per-parameter verification. The caller subsequently queries DAQ and decides whether to save a new definition with `recipe_update`, restore one with `recipe_rollback`, or dispatch a version with `recipe_apply`. These steps are separate: a trial does not save or apply a version, and endpoint verification is not a process-response measurement. Public direct-node control is closed; the bounded probe and recipe operations are the configured operation surfaces. The historical four-scene results use an earlier direct-node path and do not test current profile or binding adaptation across scenes.

d) Command verification: Command confirmation and process response answer different questions. A driver ACK is graded as `readback-verified`, `transport-ack`, or `unverified`. A matching endpoint read supports the first grade; a transport or MES application receipt supports the second. Recipe batch verification assigns `verified`, `unverified`, or `failed`; if no matching readback is available, it attempts up to three independent reads, 2 s apart. A matching retry upgrades the final ACK, while exhausted retries remain unverified. The initial ACK remains in write history and the final ACK and verdict are retained with the batch. Per-parameter receipts preserve successful outcomes when another parameter fails. A readback-verified setpoint does not establish target attainment.

e) Observation, verdict, and compensation: The caller performs recipe-path observation after batch dispatch. The October recipe case therefore compares a recipe definition, endpoint readback, and DAQ sample as separate representations. Saving a definition, applying it to equipment, and observing

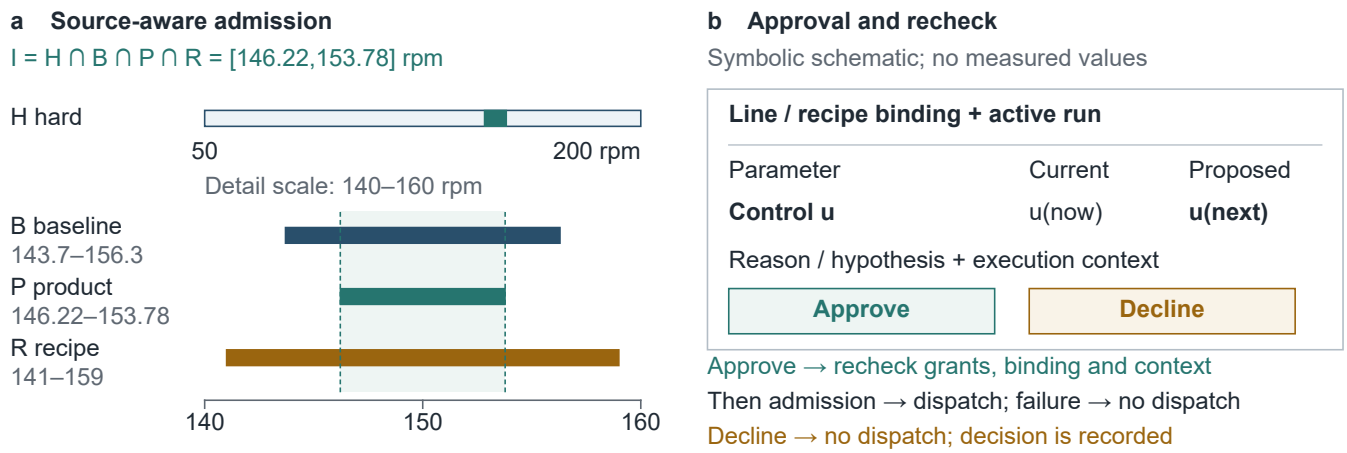


Fig. 4. Admission and approval boundaries. (a) Archived RPM example: online node interventions use the four-layer value interval of Eq. (2), whereas recipe dispatch omits the prior recipe window. (b) Symbolic human approval binds a proposed action and authority fields; execution checks run again after approval, which does not reserve equipment state.

its process response are separately evidenced operations. The evaluation does not combine these route-specific records into one universal end-to-end record.

An open node record provides a monitoring backstop. After 120 s it polls every 30 s; three cumulative samples outside the recipe monitoring window trigger a system-attributed rollback request toward the preceding value. Missing samples do not count as breaches. A caller's `keep`, `rollback`, or `uncertain` verdict records its interpretation; compensation is a new command subject to its applicable admission policy. The write audit deduplicates the same value on the same node within 10 s, while batch receipts, exploration summaries, DAQ history, and recipe versions retain their distinct lifecycles. Linking these records by actor and production context allows the command, endpoint evidence, and process observation to be inspected without treating them as interchangeable.

V. Experimental Evaluation

The evaluation examines two bounded simulator paths. R13 is an exploratory application-level protocol and data-path run; it supports only the recorded protocol, readback, lineage, and model-gate outcomes because its exact runtime source was not pinned. R20 is the primary current-version result: six sequential tasks through one registered harness across two configured scenes, with a run-specific source reconstruction kit. It supports the tested task-to-device integration path, not configuration-reuse benefit or process attainment. All process experiments use repository simulators. Older scripted and regression records remain in the companion evidence package and are not pooled with these results.

A. Evidence Units and Experimental Procedure

R13 uses two configured protocol/data paths in cast-film and annealing simulators. R20 uses the registered `omp` harness in those scenes, with three sequential task executions per scene on one simulator instance per scene. The companion reproducibility package records the older protocol, controller, deployment, direct-node, and scripted-recipe cases separately; they are not part of the current primary result.

B. Exploratory Two-Scene Protocol and Data Path (R13)

The R13 report identifies its application run as v0.8.0; the exact runtime source was not recorded. It used cast-film and annealing simulator presets. Five protocol families—OPC UA, MQTT, Modbus TCP, Modbus RTU over TCP, and HTTP/REST—passed read probes; bounded writes returned matching readbacks. Six 120-s runs per scene at 1-s sampling produced lineage-verified datasets.

For each dataset row i, let m_i be the last MV value in its history window and c_i the first CV value in its target horizon. The gate computes

$$r_\Delta(\ell) = \mathrm{corr}(\Delta m_i, \Delta c_{i+\ell}), \qquad \ell \in \{1,2,3\}, \tag{3}$$

using all rows in the dataset for which the shifted pair is defined, and selects the lag with greatest $|r_\Delta(\ell)|$. It then requires $|r_\Delta| \geq 0.2$, one-step test NRMSE ≤ 0.10, and at least 20% four-step rollout NRMSE reduction over persistence. Both post-pilot candidates failed the gates; neither produced an accepted model, profile, recommendation, or dispatch. The overlapping 1-s windows are serially dependent. This run therefore supports protocol/data-path execution and rejection behavior only, not model quality or process performance. Signal-pair selection followed an exploratory pilot, and the run-time source diff was not preserved; the R13 results are included as secondary diagnostic evidence, with detailed records in the reproducibility supplement.

Table II reports the selected signal pair, the full-dataset first-difference correlation used for eligibility screening, and two model metrics computed on held-out test rows. Both scenes failed all three criteria; these values document the recorded rejection path and are not confirmatory model-performance estimates. The archived run does not identify its exact runtime source, so the NRMSE normalization implementation cannot be verified against that run; the companion provenance notes describe the available candidate evaluator and this limit.

C. Registered-Harness Process Path

R20 ran the isolated v0.8.0 working-tree application with the registered `omp` adapter and GLM-5.3-Flash in cast-film

TABLE II
R13 EXPLORATORY MODEL-GATE DIAGNOSTICS. THE CORRELATION COLUMN IS THE FULL-DATASET ELIGIBILITY SCREEN; NRMSE AND ROLLOUT REDUCTION USE HELD-OUT TEST ROWS. BOTH CANDIDATES FAIL ALL THREE CRITERIA.

Scene	MV → CV	$\lvert r_\Delta \rvert$ (lag)	1-step NRMSE	4-step reduction
Film	`ScrewSpeedSP` → `thick`	0.0949 (2)	0.2587	14.11%
Anneal	Z3 setpoint → `StripTemp`	0.0526 (3)	0.3632	4.83%

and annealing simulator scenes. The app and simulator diffs, base revisions, dependency lockfiles, and runtime prompt assets are included with the run record; package manifests are recovered from the recorded base revisions. The external model service does not provide a deterministic replay of its selection. Each execution created a separate line, product, recipe, run, channel, agent, and task context; three sequential executions used one simulator instance per scene, so they are not independent replicates. Cast film used OPC UA for screw-speed control and MQTT for the thickness probe; annealing used Modbus TCP for both the zone-3 setpoint and strip-temperature probe. The cast-film candidate set was $\{125, 130, 135\}$ rpm and the annealing candidate set was $\{735, 740, 745\}$ °C. Given a live control read, process measurement, and target band, GLM-5.3-Flash proposed one bounded `recipe_trial`. A synthetic administrator approved each proposal; the route log recorded one trial and a 1/1 verified device receipt, and all six tasks reached the completed state. Following task completion and its evidence queries, a 20-s wait was initiated; one driver probe per task was then ingested through the isolated SQLite time-series fallback and returned under the active run tag. The integration checks cover per-execution context, an approved bounded proposal, route-specific receipt and readback, task completion, and an active-run-tagged sample; all six records contain this software path. The runner recorded 76 passing checks and no failures. The task history contains mirrored event projections, so the per-task operation count is taken from the route-specific recipe and ACK records.

Table III reports the pre-task measurement, selected setpoint, and single post-trial sample. All six post-trial values remained outside the declared target bands. This experiment verifies the bounded harness-to-simulator execution and evidence path; it does not show process attainment, convergence, or quality improvement. The single sample is not a response trajectory, and the SQLite fallback does not test the normal online DAQ backend. The two-scene run also does not isolate profile/binding changes or establish cross-harness portability.

Earlier protocol and admission-ablation suites, deterministic-controller runs, deployment regressions, direct-node missions, and a scripted recipe case are retained in the companion reproducibility package for auditability. They use older versions or interfaces and are not pooled with R13 or R20, used as matched strategy comparisons, or relied on for the current integration claims.

VI. DISCUSSION

In the exploratory R13 application run, both model candidates fail the configured eligibility and performance gates.

TABLE III
MODEL-SELECTED SETTINGS AND SIMULATOR MEASUREMENTS. EACH ROW REPORTS THE PRE-TASK VALUE, CONFIGURED TARGET BAND, MANIPULATED VARIABLE'S INITIAL-TO-SELECTED SETTING, AND ONE RUN-TAGGED SAMPLE TAKEN AFTER A 20-S WAIT INITIATED FOLLOWING TASK COMPLETION AND EVIDENCE QUERIES. ALL SIX SAMPLES ARE OUTSIDE TARGET; EXECUTIONS ARE SEQUENTIAL WITHIN EACH SCENE.

Case	Pre-task CV / target	Setting: start→trial	CV after wait
Film-1	55.82/48–52 μm	120 → 125 rpm	45.69 μm
Film-2	45.69/48–52 μm	120 → 130 rpm	46.69 μm
Film-3	46.69/48–52 μm	120 → 125 rpm	45.69 μm
Anneal-1	675.40/700–760 °C	730 → 745 °C	679.60 °C
Anneal-2	678.30/700–760 °C	730 → 745 °C	677.70 °C
Anneal-3	676.90/700–760 °C	730 → 745 °C	678.10 °C

Signal pairs were selected after a pilot, and the exact runtime source was not pinned; the retained correlations and NRMSE values therefore document the recorded gate path rather than confirmatory model performance. In R20, six sequential tasks complete a synthetically approved, readback-verified recipe trial and task closure. Each has one run-tagged sample, collected after a 20-s wait initiated after task completion and evidence queries through an isolated SQLite fallback; all six samples remain outside their target bands. This supports the tested application path, not successful process control.

R20 uses one simulator instance per scene and does not establish independent replication, convergence, or quality improvement. Its post-task sample is not a response trajectory or a fixed command-to-sample latency measurement. Controlled profile/binding adaptation, cross-harness portability, normal online DAQ backend behavior, and physical transfer remain untested. The synthetic approval records do not measure operator effectiveness [27]–[29]; restricted tool access does not establish deployment security [22], [30], [31] or replace independently engineered PLC/SIS protection [32].

Further evaluation needs a versioned build, matched configuration changes, independent simulator replications, normal online DAQ, and physical-plant trials.

VII. CONCLUSION

AgentWorkShop provides an implemented task-to-device integration architecture that joins collaborative harnesses, production context, operation checks, semantic I/O, and route-specific records. R13 and R20 demonstrate selected simulator paths: R13's model candidates fail their gates, and all six R20 samples remain outside target. The evidence establishes integration behavior and does not establish process-control efficacy.

REFERENCES

[1] Y. Xia, N. Jazdi, J. Zhang, C. Shah, and M. Weyrich, "Control industrial automation system with large language model agents," in *Proc. IEEE 30th Int. Conf. on Emerging Technologies and Factory Automation (ETFA)*, 2025, pp. 1–8, doi: 10.1109/ETFA65518.2025.11205539.

[2] K.-C. Huang, H.-T. Lin, C.-H. Lin, and M.-T. Huang, "Large language model integration in industrial cyber-physical systems: A multi-modal architecture for automated control generation," in *Proc. Int. Conf. on Control, Automation and Diagnosis (ICCAD)*, 2026, pp. 1–6, doi: 10.1109/ICCAD69956.2026.11643124.

[3] K. Köhle, D. Anicic, T. A. Runkler, and R. Graf, "LLM-driven multi-agent control for skill-based smart manufacturing," in *Proc. IEEE 31st Int. Conf. on Emerging Technologies and Factory Automation (ETFA)*, 2026, pp. 1–8, doi: 10.1109/ETFA68377.2026.11725257.

[4] M. Waseem, K. Bhatta, C. Li, and Q. Chang, "Pretrained LLMs as real-time controllers for robot operated serial production line," *Mach. Learn.: Eng.*, vol. 2, no. 1, 2026, art. no. 015006; doi: 10.1088/3049-4761/ae4ee8.

[5] Z. Liu *et al.*, "Agents4PLC: Automating closed-loop PLC code generation and verification in industrial control systems using LLM-based agents," *IEEE Trans. Softw. Eng.*, vol. 52, no. 5, pp. 1672–1687, 2026, doi: 10.1109/TSE.2026.3667895.

[6] R. Wu, J. Ai, and T. Li, "InstructMPC: A human-LLM-in-the-loop framework for context-aware control," in *Proc. 2025 IEEE 64th Conference on Decision and Control (CDC)*, 2025, pp. 172–179, doi: 10.1109/CDC57313.2025.11312156.

[7] J. Lim and I. Kovalenko, "A large language model-enabled control architecture for dynamic resource capability exploration in multi-agent manufacturing systems," in *Proc. IEEE 21st Int. Conf. on Automation Science and Engineering (CASE)*, 2025, pp. 2088–2095, doi: 10.1109/CASE58245.2025.11163802.

[8] Z. Zhao *et al.*, "A large language model-based multi-agent manufacturing system for intelligent shopfloors," *Adv. Eng. Inform.*, vol. 69, 2026, art. no. 103888; doi: 10.1016/j.aei.2025.103888.

[9] M. S. Gill, J. Vyas, A. Markaj, F. Gehlhoff, and M. Mercangöz, "Leveraging LLM agents and digital twins for fault handling in process plants," in *Proc. IEEE ETFA*, 2025, pp. 1–8, doi: 10.1109/ETFA65518.2025.11205597.

[10] B. Hofmann, N. Piechulek, S. Kreitlein, J. Franke, and P. Bruendl, "Beyond touch-based human-machine interface: Control your machines in natural language by utilizing large language models and OPC UA," *arXiv preprint arXiv:2510.11300*, 2025. [Online]. Available: https://arxiv.org/abs/2510.11300v2

[11] A. Biskupovic, A. Villalonga, F. Castaño, R. E. Haber, and F. Núñez, "Agent-based supervision for service-oriented industrial cyber–physical systems," *IEEE Trans. Ind. Inform.*, vol. 21, no. 3, pp. 2719–2728, 2025, doi: 10.1109/TII.2024.3514178.

[12] L. M. Vieira da Silva, A. Köcher, and F. Gehlhoff, "Beyond formal semantics for capabilities and skills: Model context protocol in manufacturing," in *2025 IEEE 30th International Conference on Emerging Technologies and Factory Automation (ETFA)*, 2025.

[13] K. Köhle, D. Anicic, T. A. Runkler, K. Dorofeev, A. S. Thuluva, and R. Graf, "Virtually commissioned semantic recipes based on opc ua skills," in *2025 IEEE 8th International Conference on Industrial Cyber-Physical Systems (ICPS)*, 2025, pp. 1–6.

[14] X. Jiang *et al.*, "Agentic AI for safety-aware process monitoring and fault diagnosis: A review," *Processes*, vol. 14, no. 13, 2026, art. no. 2112; doi: 10.3390/pr14132112.

[15] Model Context Protocol contributors, "Model Context Protocol: Tools," Specification, version 2026-07-28, 2026, accessed: Oct. 20, 2026. [Online]. Available: https://modelcontextprotocol.io/specification/2026-07-28/server/tools

[16] ISA, "ISA-88.00.01-2010: Batch control—part 1: Models and terminology," International Standard, 2010.

[17] ——, "ANSI/ISA-95.00.01-2025 (IEC 62264-1 mod): Enterprise-control system integration—part 1: Models and terminology," International Standard, 2025.

[18] J. Moyne, E. del Castillo, and A. M. Hurwitz, Eds., *Run-to-Run Control in Semiconductor Manufacturing*, 1st ed. Boca Raton, FL: CRC Press, 2000.

[19] H. Wang, C. M. Poskitt, and J. Sun, "AgentSpec: Customizable runtime enforcement for safe and reliable LLM agents," in *Proc. IEEE/ACM 48th Int. Conf. on Software Engineering (ICSE)*, 2026, pp. 2938–2950, doi: 10.1145/3744916.3764546.

[20] E. Debenedetti *et al.*, "Defeating prompt injections by design," 2025, arXiv preprint arXiv:2503.18813. [Online]. Available: https://arxiv.org/abs/2503.18813

[21] U.S. Food and Drug Administration, "Part 11, electronic records; electronic signatures—scope and application," Guidance for Industry, 2003.

[22] IEC, "IEC 62443-3-3:2013: Industrial communication networks—network and system security—part 3-3: System security requirements and security levels," Standard, 2013.

[23] IEEE, "IEEE std 1815-2012: Standard for electric power systems communications—distributed network protocol (DNP3)," Standard, 2012.

[24] M. Alshiekh, R. Bloem, R. Ehlers, B. Könighofer, S. Niekum, and U. Topcu, "Safe reinforcement learning via shielding," in *Proc. AAAI Conference on Artificial Intelligence*, vol. 32, no. 1, 2018, pp. 2669–2678, doi: 10.1609/aaai.v32i1.11797.

[25] A. D. Ames, X. Xu, J. W. Grizzle, and P. Tabuada, "Control barrier function based quadratic programs for safety critical systems," *IEEE Trans. Autom. Control*, vol. 62, no. 8, pp. 3861–3876, 2017, doi: 10.1109/TAC.2016.2638961.

[26] L. Sha, "Using simplicity to control complexity," *IEEE Softw.*, vol. 18, no. 4, pp. 20–28, 2001, doi: 10.1109/MS.2001.936213.

[27] R. Parasuraman and D. H. Manzey, "Complacency and bias in human use of automation: An attentional integration," *Hum. Factors*, vol. 52, no. 3, pp. 381–410, 2010, doi: 10.1177/0018720810376055.

[28] R. Parasuraman, T. B. Sheridan, and C. D. Wickens, "A model for types and levels of human interaction with automation," *IEEE Trans. Syst., Man, Cybern., Part A*, vol. 30, no. 3, pp. 286–297, 2000, doi: 10.1109/3468.844354.

[29] L. Bainbridge, "Ironies of automation," *Automatica*, vol. 19, no. 6, pp. 775–779, 1983, doi: 10.1016/0005-1098(83)90046-8.

[30] K. Greshake, S. Abdelnabi, S. Mishra, C. Endres, T. Holz, and M. Fritz, "Not what you've signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection," in *Proc. 16th ACM Workshop on Artificial Intelligence and Security*, 2023, pp. 79–90, doi: 10.1145/3605764.3623985.

[31] Y. Liu, Y. Jia, R. Geng, J. Jia, and N. Z. Gong, "Formalizing and benchmarking prompt injection attacks and defenses," in *Proc. 33rd USENIX Security Symposium (USENIX Security 24)*, 2024, pp. 1831–1847. [Online]. Available: https://www.usenix.org/conference/usenixsecurity24/presentation/liu-yupei

[32] IEC, "IEC 61511-1:2016: Functional safety—safety instrumented systems for the process industry sector—part 1: Framework, definitions, system, hardware and application programming requirements," International Standard, 2016.